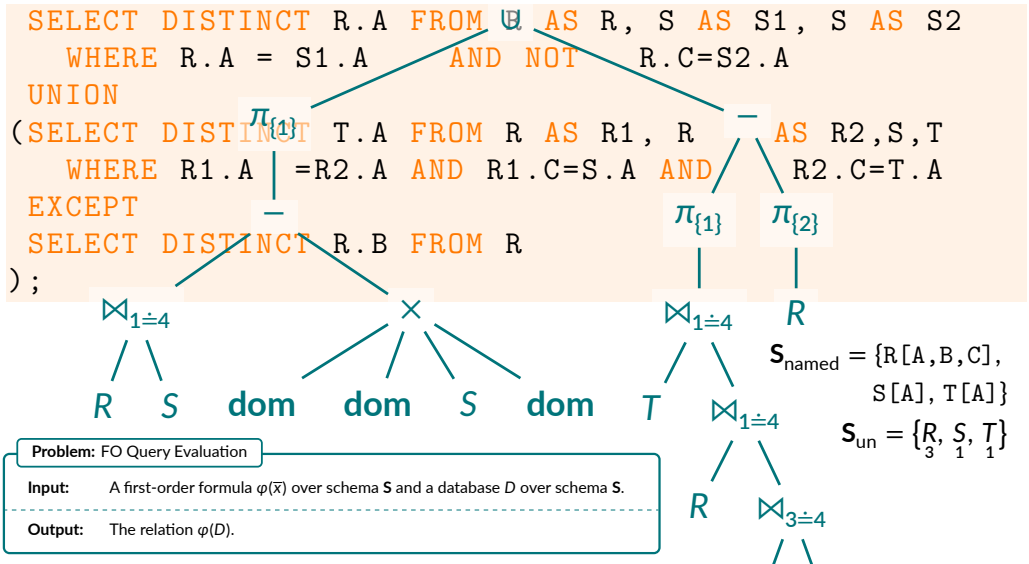
Theoretical Foundations of Data Management

Prof. Dr. Christoph Berkholz

Winter term 2026/27

TECHNISCHE UNIVERSITÄT
ILMENAU



Problem: FO Query Evaluation

Input: A first-order formula $\varphi(\bar{x})$ over schema $\mathbf{S}$ and a database D over schema $\mathbf{S}$.

Output: The relation $\varphi(D)$.

$$\varphi(x) = \exists y \exists z \left(R(x,y,z) \wedge S(x) \wedge \neg S(z) \right) \vee \left(\exists y \exists z \left(T(x) \wedge R(y,z,x) \wedge \exists x \exists z \left(S(x) \wedge R(y,z,x) \right) \right) \wedge \forall y \forall z \neg R(y,x,z) \right)$$

Preface

This document contains slides and further notes for the course *Theoretical Foundations of Data Management*. This document **does not contain everything that is relevant for the course and the exam**. Relevant for the final exam is everything that was taught and discussed in lecture and exercise classes in the respective semester.

Copyright. The lecture notes are published under the *Creative Commons Attribution-ShareAlike 4.0 International Public License (CC BY-SA 4.0)*. The legally binding license agreement can be found at `https://creativecommons.org/licenses/by-sa/4.0/`.

Literature & Sources The lecture is based to a large extend on (the 2021 draft version of) the book *Arenas, Barceló, Libkin, Martens, Pieris: Database Theory* [ABLMP22], which is freely available online at `https://github.com/pdm-book/community`. Another classical textbook on that topic is *Abiteboul, Hull, Vianu: Foundations of Databases* [AHV95]. If applicable, picture credits are provided at the end of this document.

Christoph Berkholz

Version: 19th October, 2026

The lecture notes are currently in a preliminary status. Use them with a critical mindset and check out the moodle course for updates.

Contents

I Relational Databases and Query Languages **5**

1 The Relational Model **5**
1.1 The named perspective on relational databases 5
1.2 Queries and query languages 6
1.3 Relational Algebra 6

2 SQL and Relational Algebra **8**
2.1 SQL 8
2.2 RA equals coreSQL 10

3 The Unnamed Perspective **12**
3.1 Relational model in the unnamed perspective 12
3.2 Unnamed RA 13

4 First-order logic **14**
4.1 First-order logic and relational algebra 16

Literatur **17**

Part I
Relational Databases and Query Languages

1 The Relational Model

slide 1.1

1.1 The named perspective on relational databases

Relational databases: the basics

slide 1.2

To study and prove things about databases and query languages, we need formal defintions of these objects. This can be done in several ways, we will focus on the following two:

- The named perspective (this lecture).
- The unnamed perspective (later).

We fix the following countably infinite disjoint sets:

- *Rel* ...relation names
- *Att* ...attribute names
- *Const* ...values (database entries)

Note: *Const* = {`Alice`, `Bob`, `1`, `2`, `3`, `'a'`, `3.14`, ...} contains *all* possible entries in a database. In this course, we do not distinguish data types as `integer`, `float`, `string`, `char`, ...

Database tuples and relations

slide 1.3

A ***database tuple*** is a function $t : U \to Const$, where $U = \{A_1, \dots, A_k\} \subseteq Att$ is finite.

- The set U is the *sort* of t.
- If $|U| = k$, then t is called *k-ary* (0-ary = nullary, 1-ary = unary, 2-ary = binary, 3-ary = ternary)
- By $t = (A_1 : a_1, \dots, A_k : a_k)$ we denote the tuple with $t(A_i) = a_i$. Note that the order does not matter, e. g. $(A : a, B : b) = (B : b, A : a)$ as they are the same function.

A ***database relation*** is a finite set S of database tuples of the same sort U.

- The set U is the *sort* of S, denoted by $sort(S)$.
- S is *k-ary* if $|sort(S)| = k$.
- *nDBR* is the set of all (named) database relations.

Database schemas and Databases

slide 1.4

A ***named database schema*** is a partial function $\boldsymbol{S} : Rel \to \mathcal{P}_{\text{fin}}(Att)$ with finite domain $dom(\boldsymbol{S})$.[1]

- We call $\boldsymbol{S}(R)$ the *sort* of the relation name R.
- The *arity* of the relation name R under $\boldsymbol{S}$ is $ar_{\boldsymbol{S}}(R) := |\boldsymbol{S}(R)|$.
- We write $R[A_1, \dots, A_k]$ to denote $\boldsymbol{S}(R) = \{A_1, \dots, A_k\}$. (Again, the order of attributes does not matter: $R[A, B]$ and $R[B, A]$ denote the same thing.)
- To specify a schema we also write, e. g., $\boldsymbol{S} = \{R[A, B], S[B], T[A, B, C]\}$

A ***database*** D of schema $\boldsymbol{S}$ is a function $D : dom(\boldsymbol{S}) \to nDBR$, such that $sort(D(R)) = \boldsymbol{S}(R)$ for all $R \in dom(\boldsymbol{S})$.

Discussion of the formal model

slide 1.5

The named perspective on databases has in particular the following properties:

- There is no order of the "columns" in a database relation.
- There is no order of the "rows" in a database relation.
- Attributes have no "data types".
- A database relation is a *set* of tuples (and not a list or a multiset). Therefore it cannot contain the same tuple twice.
- We do not allow `NULL`-values in the tuples.

1.2 Queries and query languages

slide 1.6

Queries

- Informally, a query is a request to get some information out of a database.
- In this lecture we require that the output is a (finite) database relation.

Let $k \in \mathbb{N} = \{0, 1, 2, 3, \ldots\}$.
A *k-ary* ***query function*** *q over schema* $\boldsymbol{S}$ is a function $q : DB(\boldsymbol{S}) \to nDBR$, where

- $DB(\boldsymbol{S})$ is the set of all databases over schema $\boldsymbol{S}$ and
- every relation $q(D) \in nDBR$ has the same sort U with $|U| = k$.

Note that 0-ary query functions q map the database D to either

- the 0-ary empty relation $q(D) = \emptyset$, which we also interpret as *false*, or
- the 0-ary relation $q(D) = \{()\}$ that contains the empty tuple, which we interpret as *true*.

While this function (with infinite domain) fully specifies the *sematics* of a query, we need to specify them as finite *syntactic* objects.

slide 1.7

Query languages

Let Σ be some finite alphabet (think of `ASCII` or `UTF-8`).

A ***query language*** is a subset $L \subseteq \Sigma^*$ of ***query expressions*** such that:

- L has a decidable syntax. That is, there is an algorithm that receives a string $\varphi \in \Sigma^*$ and decides whether φ is a valid query expression (i. e. $\varphi \in L$).
- L has a decidable sematics. That is, for every query expression $\varphi \in L$ there is unique query function q_φ and there is an algorithm that receives $\varphi \in L$ and a database D and computes the relation $q_\varphi(D)$.

In the following, we use ***query*** for both, query expressions and (their associated) query function.

slide 1.8

Expressiveness of query languages

The *expressiveness* of a query language L is given by the set $\{q_\varphi \mid \varphi \in L\}$ of query functions that can be expressed in this language.

This allows to compare the power of query languages:

- We say that L is at least as expressive as L' ($L \geqslant L'$) if $\{q_\varphi \mid \varphi \in L\} \supseteq \{q_\varphi \mid \varphi \in L'\}$.
- We say that L and L' are equally expressive ($L \equiv L'$) if $L \geqslant L'$ and $L' \geqslant L$.

Typically there is a tradeoff between

1. The *expressiveness* of a query language.
2. The *complexity* of query evaluation algorithms.

1.3 Relational Algebra

slide 1.9

Relational Algebra: conditions (syntax)

Let $U \subseteq Att$. A *condition over* U is a Boolean combination of statements of the form $A \doteq B$ or $A \doteq a$ for $A, B \in U$ and $a \in Const$.

To be more precise, we give the following *recursive definition*:

Base cases

- $A \doteq B$ is a condition over U for all $A, B \in U$.
- $A \doteq a$ is a condition over U for all $A \in U$ and $a \in Const$.

Recursive cases

- If θ is a condition over U, then $\neg\theta$ is a condition over U.
- If θ_1 and θ_2 are conditions over U, then $(\theta_1 \vee \theta_2)$ is a condition over U.
- If θ_1 and θ_2 are conditions over U, then $(\theta_1 \wedge \theta_2)$ is a condition over U.

Relational Algebra: conditions (semantics)

slide 1.10

Let $U \subseteq Att$. For every condition θ over U and tuple t whose sort contains U we define the relation *"t satisfies θ ($t \models \theta$)"*.

This can be done recursively over the recursive definition of conditions.

Base cases

- $t \models A \doteq B$ if $t(A) = t(B)$.
- $t \models A \doteq a$ if $t(A) = a$.

Recursive cases

- $t \models \neg\theta$ if it is not the case that $t \models \theta$
- $t \models (\theta_1 \vee \theta_2)$ if $t \models \theta_1$ or $t \models \theta_2$ (or both)
- $t \models (\theta_1 \wedge \theta_2)$ if $t \models \theta_1$ and $t \models \theta_2$

Relational Algebra: Syntax

slide 1.11

Let $\boldsymbol{S}$ be a schema. We recursively define *RA expressions of sort U (over schema $\boldsymbol{S}$)*:

Base cases

- R is a RA-expression of sort $\boldsymbol{S}(R)$ for all $R \in dom(\boldsymbol{S})$.
- $\{(A : a)\}$ is a RA-expression of sort $\{A\}$ for all $A \in Att$ and $a \in Const$.

Recursive cases

- **(selection)** If e is a RA-expression of sort U and θ is a condition over U, then $\sigma_\theta(e)$ is a RA-expression of sort U.
- **(projection)** If e is a RA-expression of sort U and $\alpha \subseteq U$, then $\pi_\alpha(e)$ is a RA-expression of sort α.
- **(join)** If e_1 and e_2 are RA-expressions of sort U_1 and U_2 (resp.), then $(e_1 \bowtie e_2)$ is a RA-expression of sort $U_1 \cup U_2$.
- **(rename)** If e is a RA-expression of sort U, $A \in U$, and $B \in Att \setminus U$, then $\rho_{A\to B}(e)$ is a RA-expression of sort $(U \setminus \{A\}) \cup \{B\}$.
- **(union / difference)** If e_1 and e_2 are RA-expression of sort U, then $(e_1 \cup e_2)$ and $(e_1 - e_2)$ are RA-expressions of sort U.

Relational Algebra: Semantics

slide 1.12

Let $\boldsymbol{S}$ be a schema. For every RA expression e of sort U over $\boldsymbol{S}$ we define the query function $q_e(D)$ applied to database D over $\boldsymbol{S}$ recursively over the definition of RA expressions:

Base cases

- If $e = R$, then $q_e(D) := D(R)$.
- If $e = \{(A : a)\}$, then $q_e(D) := \{(A : a)\}$.

We need some further notation:

- For $t : U \to Att$ and $\alpha \subseteq U$ we let $t \restriction_\alpha$ be the tuple of sort α with $t \restriction_\alpha (A) := t(A)$ for all $A \in \alpha$.
- For $t : U \to Att$, $A \in U$, and $B \notin U$ we let $t_{A\to B}$ be the tuple of sort $(U \setminus A) \cup \{B\}$ with $t_{A\to B}(B) := t(A)$ and $t_{A\to B}(C) := t(C)$ for all $C \in (U \setminus A)$.

Recursive cases

- If $e = \sigma_\theta(e')$, then $q_e(D) := \{t \in q_{e'}(D) \mid t \models \theta\}$.
- If $e = \pi_\alpha(e')$, then $q_e(D) := \{t \restriction_\alpha \mid t \in q_{e'}(D)\}$.
- If $e = (e_1 \bowtie e_2)$ and $sort(e_1) = U_1$ and $sort(e_2) = U_2$, then $q_e(D) := \{t \mid sort(t) = U_1 \cup U_2$ and $t \restriction_{U_1} \in q_{e_1}(D)$ and $t \restriction_{U_2} \in q_{e_2}(D)\}$.
- If $e = \rho_{A\to B}(e')$, then $q_e(D) := \{t_{A\to B} \mid t \in q_{e'}(D)\}$.
- If $e = (e_1 \cup e_2)$, then $q_e(D) := q_{e_1}(D) \cup q_{e_2}(D)$.

- If $e = (e_1 - e_2)$, then $q_e(D) := q_{e_1}(D) \setminus q_{e_2}(D)$.

slide 1.13

Examples

MODULES

Number	Title	ETCS	Language	Unit
201312	TFoDM	5	english	2242
200062	AuD1	5	german	2242
200064	AuD2	5	german	2242
201114	BK	5	german	2242
201171	AuK 1+2	10	german	2242
200080	Algorithms	5	english	2242
200065	EA	5	german	2242
201196	CSP	5	german	2242
...	...	...	...	...

COURSES

Name	Type	Module	Semester	Lecturer
TFoDM	lecture	201312	WS26/27	Berkholz
TFoDM	exercise	201312	WS26/27	Irwin
AuD2	exercise	200064	WS26/27	Micun
CSP	lecture	201196	WS25/26	Berkholz
Algorithms	lecture	200080	SoSe26	Berkholz
Algorithms	exercise	200080	SoSe26	Merz
EA	lecture	200065	WS26/27	Berkholz
EA	exercise	200065	WS26/27	Micun
...	...	...	...	...

PARTICIPANTS

ID	Course	Type	Semester
098693	TFoDM	lecture	WS26/27
098693	TFoDM	exercise	WS26/27
493677	TFoDM	exercise	WS26/27
...	...	...	...

STUDENTS

ID	Name	Email	Study program
121815	Ada Lovelace	ada@tu-ilmenau.de	MA_IN
031913	Paul Erdős	paul@tu-ilmenau.de	BA_MA
121887	Srinivasa Ramanujan	srinivasa@tu-ilmenau.de	MA_DS
...	...	...	...

MODULES

Number	Title	ETCS	Language	Unit
...	...	...	...	...

COURSES

Name	Type	Module	Semester	Lecturer
...	...	...	...	...

PARTICIPANTS

ID	Course	Type	Semester
...	...	...	...

STUDENTS

ID	Name	Email	Study program
...	...	...	...

Names and email addresses of all students attending a course lectured by Oliver Irwin.

$$\pi_{\{\text{Name, Email}\}}\Big(\pi_{\text{ID}}(\sigma_{\text{Lecturer} \doteq \text{Irwin}}\text{Courses} \bowtie \rho_{\text{Course}\to\text{Name}}\text{Participants}) \bowtie \text{Students}\Big)$$

Email of all students registered for either the lecture or the exercise of TFoDM in WS26/27.

$$e_1 := \pi_{\text{ID}}\sigma_{\text{Course} \doteq \text{TFoDM} \wedge \text{Type} \doteq \text{lecture} \wedge \text{Semester} \doteq \text{WS26/27}}\ \text{Participants}$$

$$e_2 := \pi_{\text{ID}}\sigma_{\text{Course} \doteq \text{TFoDM} \wedge \text{Type} \doteq \text{exercise} \wedge \text{Semester} \doteq \text{WS26/27}}\ \text{Participants}$$

$$e := \pi_{\{\text{Name, Email}\}}\Big(((e_1 - e_2) \cup (e_2 - e_1)) \bowtie \text{Students}\Big)$$

Does the Algorithms group (2242) offer courses in german?

$$\pi_{\varnothing}\ \sigma_{\text{Unit} \doteq 2242 \wedge \text{Language} \doteq \text{german}}\ \text{Modules}$$

Is every data science master student registered for TFoDM?

$$\underbrace{\pi_{\varnothing}\{(\text{ID}:\ 0)\}}_{=\{()\}} - \pi_{\varnothing}\Big(\underbrace{\underbrace{\pi_{\text{StudID}}\sigma_{\text{St. pr.} \doteq \text{MA_DS}}\ \text{Students}}_{\text{all MA_DS students}} - \underbrace{\pi_{\text{StudID}}\sigma_{\text{Course} \doteq \text{TFoDM}}\ \text{Participants}}_{\text{all TFoDM participants}}}_{\text{all MA_DS students not attending TFoDM}}\Big)$$

$$\underbrace{\qquad\qquad}_{\substack{\{\} \text{ if all MA_DS students attend TFoDM} \\ \{()\} \text{ if not all MA_DS students attend TFoDM}}}$$

$$\underbrace{\qquad\qquad}_{\substack{\{()\} \text{ if all MA_DS students attend TFoDM} \\ \{\} \text{ if not all MA_DS students attend TFoDM}}}$$

slide 2.1

2 SQL and Relational Algebra

2.1 SQL

slide 2.2

coreSQL: syntax

Let $\boldsymbol{S}$ be a schema, $U \subseteq Att$ and (for technical reasons) $\vec{U}$ an ordered list of elements from U.
We recursively define *coreSQL expressions of sort* $\vec{U}$ *(over schema* $\boldsymbol{S}$*)*.

Base cases

```
SELECT DISTINCT <s-list>
  FROM <f-list>
```

and

```
SELECT DISTINCT <s-list>
  FROM <f-list>
  WHERE <condition>
```

is a coreSQL expression `Q` of sort $\vec{U}$;

- for details on `<s-list>`, `<f-list>`, `<condition>`, and $\vec{U}$ see next slides,
- these are also *recursive* cases if `<f-list>` contains subqueries.

Recursive cases
If `Q1` and `Q2` are coreSQL expressions of the same sort $\vec{U}$, then `(Q1 UNION Q2)` and `(Q1 EXCEPT Q2)` are coreSQL expressions of sort $\vec{U}$.

coreSQL: `<f-list>`

slide 2.3

```
SELECT DISTINCT <s-list>
  FROM <f-list>
  WHERE <condition>
```

The `<f-list>` is a comma separated list of expressions of the form

- `R AS S` where `R` $\in$ *dom*($\boldsymbol{S}$) is a relation name in the schema $\boldsymbol{S}$ and `S` $\in$ *Rel* $\setminus$ *dom*($\boldsymbol{S}$) is a new relation name serving as *alias*
- `Q1 AS S` where `Q1` is a coreSQL expression and `S` $\in$ *Rel* $\setminus$ *dom*($\boldsymbol{S}$) is a new relation name serving as alias

We require all aliasses in the `<f-list>` to be distinct.

The alias relation names inheret the sort $\vec{U}$ of their corresponding relation name / coreSQL expression.

coreSQL: `<s-list>`

slide 2.4

```
SELECT DISTINCT <s-list>
  FROM <f-list>
  WHERE <condition>
```

The `<s-list>` is a comma separated list of expressions of the form

- `a AS A` where `A` $\in$ *Att* is an attribute alias and $a \in$ *Const* is a constant (for example: `17 AS A` or `'Goethe' AS B`)
- `R.B AS A` where `R` $\in$ *Rel* is an alias with sort $\vec{U}$ mentioned in the `<f-list>`, `B` $\in U$ is an attribute of `R` and `A` $\in$ *Att* is an attribute alias.

We require all attribute aliasses in the `<s-list>` to be distinct.

If `<s-list>` = `X1 AS A1, ..., Xk AS Ak` then the *sort* of SQLcore expression is $\vec{U}$:= (`A1`, ..., `Ak`).

coreSQL: `<condition>`

slide 2.5

```
SELECT DISTINCT <s-list>
  FROM <f-list>
  WHERE <condition>
```

The `<condition>` is closely related to the selection condition θ in relational algebra:

Base cases

- `R1.A = R2.B` is a `<condition>` if `R1`, `R2` ∈ *Rel* are relation aliasses from the `<f-list>` of sorts $\vec{U_1}, \vec{U_2}$ with attributes `A` ∈ U_1 and `B` ∈ U_2.
- `R1.A = a` is a `<condition>` for `R1.A` as above and `a` ∈ *Const*.

Recursive cases

- If `c` is a `<condition>`, then `NOT c` is a `<condition>`.
- If `c1` and `c2` are a `<condition>`, then `(c1 OR c2)` is a `<condition>`.
- If `c1` and `c2` are a `<condition>`, then `(c1 AND c2)` is a `<condition>`.

slide 2.6

coreSQL: semantics

coreSQL is a proper syntactical fragment of the SQL standard (ISO/IEC 9075)

the semantics of the coreSQL fragment is the same as for SQL
(which is not formally defined and details depend on the engine (mySQL/postgreSQL/...))

we leave a formal mathematical definition of the query function $q_{\mathtt{Q}}$ as an exercise

2.2 RA equals coreSQL

slide 2.7

From coreSQL to RA

- We define for every coreSQL expression `Q` an RA expression $e_{\mathtt{Q}}$ that computes the same query function.
- This corresponds (in a very simplified form) to what a database management system does when excecuting a SQL query:
  1. The SQL query is parsed and translated to RA
  2. The query optimizer rewrites the RA expression to obtain an optimzed plan.
  3. The optimized plan is excecuted.
- At this point we focus on 1. and will later discuss aspects of 2. and 3. .
- Our goal is to prove that coreSQL and RA are equally expressive query languages (and the translation from coreSQL to RA is one direction).
- Again, the translation is done recursively over the recursive definition of coreSQL expressions.

slide 2.8

From coreSQL to RA

First assume the coreSQL expression `Q` has the form

```
SELECT DISTINCT S_{i_1}.A_{k_1} AS B_1, ..., S_{i_ℓ}.A_{k_ℓ} AS B_ℓ, a_1 AS C_1, ..., a_m AS C_m
  FROM Q_1 AS S_1, ..., Q_n AS S_n
  WHERE <condition>
```

where each $\mathtt{Q}_j$ is either a relation name from $\boldsymbol{S}$ and $e_{\mathtt{Q}_j} := \mathtt{Q}_j$or $\mathtt{Q}_j$ is a subquery and $e_{\mathtt{Q}_j}$ has been defined recursively.Then

$$e_{\mathtt{Q}} := \rho_{\text{out}}\Big(\pi_{\alpha}\big(\sigma_{\theta}(\rho_1(e_{\mathtt{Q}_1}) \bowtie \cdots \bowtie \rho_n(e_{\mathtt{Q}_n}))\big) \bowtie \{(\mathtt{C}_1 : \mathtt{a}_1)\} \bowtie \cdots \bowtie \{(\mathtt{C}_m : \mathtt{a}_m)\}\Big)$$

where

- $\alpha = \{\mathtt{S}_{i_1}.\mathtt{A}_{k_1}, \ldots, \mathtt{S}_{i_\ell}.\mathtt{A}_{k_\ell}\}$
- θ is the natural translation of `<condition>` to RA syntax
- each ρ_i is a sequence of $\rho_{\mathtt{A}\to\mathtt{S}_i.\mathtt{A}}$ for every attribute `A` of $\mathtt{Q}_i$
- ρ_{out} renames each $\mathtt{S}_{i_s}.\mathtt{A}_{k_s}$ to $\mathtt{B}_s$

If `Q` = `(Q1 UNION Q2)` then $e_{\mathtt{Q}} := (e_{\mathtt{Q}1} \cup e_{\mathtt{Q}2})$.

If `Q` = `(Q1 EXCEPT Q2)` then $e_{\mathtt{Q}} := (e_{\mathtt{Q}1} - e_{\mathtt{Q}2})$

slide 2.9

From RA to coreSQL

Let $\boldsymbol{S}$ be a schema. We define for every RA expression e of sort U a coreSQL expression $\mathtt{Q}_e$ of sort $\vec{U}$ that computes the same query function.

Base cases

- If $e = R$ of sort $U = \boldsymbol{S}(R) = \{A_1, \ldots, A_k\}$, then

  $Q_e :=$
  ```
  SELECT DISTINCT S.A1 AS A1, ..., S.Ak AS Ak
    FROM R AS S
  ```

  (In SQL we could also just write `SELECT * FROM R`)
- If $e = \{(A : a)\}$, then $Q_e :=$ `SELECT a AS A`.

From RA to coreSQL

slide 2.10

Recursive case: projection

Let $e = \pi_\alpha(e')$ and with $\alpha = \{A_1, \ldots, A_s\}$.

$Q_e :=$
```
SELECT DISTINCT S.A1 AS A1, ..., S.As AS As
  FROM Qe' AS S
```

Recursive case: selection

Let $e = \sigma_\theta(e')$ and assume that $Q_{e'}$ is of sort $\vec{U} = (A_1, \ldots, A_k)$.

$Q_e :=$
```
SELECT DISTINCT S.A1 AS A1, ..., S.Ak AS Ak
  FROM Qe' AS S
  WHERE <condition>
```

Where `<condition>` is the condition θ translated to coreSQL syntax and where every A_ℓ is replaced by $S.A_\ell$.

From RA to coreSQL

slide 2.11

Recursive case: rename

Let $e = \rho_{A_\ell \to B}(e')$ and assume that $Q_{e'}$ is of sort $\vec{U} = (A_1, \ldots, A_k)$.

$Q_e :=$
```
SELECT DISTINCT S.A1 AS A1, ..., S.Aℓ-1 AS Aℓ-1,
                S.Aℓ AS B,
                S.Aℓ+1 AS Aℓ+1..., S.Ak AS Ak
  FROM Qe' AS S
```

Recursive cases: union and difference

Let $e = (e_1 \cup e_2)$ and assume that $sort(e_1) = sort(e_2) = \{A_1, \ldots, A_k\}$.

$Q_e :=$
```
(
  SELECT DISTINCT S.A1 AS A1, ..., S.Ak AS Ak
    FROM Qe1 AS S
UNION
  SELECT DISTINCT T.A1 AS A1, ..., T.Ak AS Ak
    FROM Qe2 AS T
)
```

If $e = (e_1 - e_2)$, then as above with `EXCEPT` instead of `UNION`.

From RA to coreSQL

slide 2.12

Recursive case: join

Let $e = (e_1 \bowtie e_2)$ and assume that $sort(e_1) = \{A_1, \ldots, A_k, B_1, \ldots, B_n\}$ and $sort(e_2) = \{B_1, \ldots, B_n, C_1, \ldots, C_m\}$.

$Q_e :=$
```
SELECT DISTINCT S.A1 AS A1, ..., S.Ak AS Ak,
                S.B1 AS B1, ..., S.Bn AS Bn,
                T.C1 AS C1, ..., T.Cm AS Cm
  FROM Qe1 AS S, Qe2 AS T
  WHERE S.B1 = T.B1 AND ... AND S.Bn = T.Bn
```

This finishes the proof that RA and coreSQL are equally expressive:

Theorem 2.1. RA ≡ coreSQL

slide 3.1

3 The Unnamed Perspective

3.1 Relational model in the unnamed perspective

slide 3.2

The unnamed perspective

So far, we formalized databases and queries using the *named perspective*.

Sometimes, in particular for mathematical proofs and logical characterizations, the related but different *unnamed perspective* fits better.

Again we use the following countably infinite disjoint sets:

- *Rel* ...relation names
- *Const* ...values (database entries)

(*Att* is not needed here)

slide 3.3

Database tuples and relations

An ***unnamed database tuple*** is an element $t \in Const^k$ for some $k \in \mathbb{N}$.

- Examples: $(\texttt{Bob}, 17)$, $(\texttt{Ilmenau}, 98693, 03677)$, (12), $()$
- Again, k is the *arity* of t.
- Note that the order *does* matter, e. g. $(\texttt{Bob}, 17) \neq (17, \texttt{Bob})$.

We also write $\overline{a} = (a_1, \dots, a_k)$ for (unnamed database) tuples $\overline{a}$.

An ***unnamed database relation*** of arity $k \in \mathbb{N}$ is a finite set S of unnamed database tuples of arity k.

- *uDBR* is the set of all unnamed database relations.

slide 3.4

Database schemas and databases

An ***unnamed database schema*** is a partial function $\boldsymbol{S} : Rel \to \mathbb{N}$ with finite domain $dom(\boldsymbol{S})$.

- $\boldsymbol{S}(R)$ just fixes the *arity* of the relation name R.
- We also write $\boldsymbol{S} = \{R[2], S[3], T[1]\}$, where $R[r]$ means that R gets assigned the arity $\boldsymbol{S}(R) = r$ under $\boldsymbol{S}$.

A ***database*** D of (unnamed) schema $\boldsymbol{S}$ is a function $D : dom(\boldsymbol{S}) \to uDBR$, such that $ar(D(R)) = \boldsymbol{S}(R)$ for all $R \in dom(\boldsymbol{S})$.

slide 3.5

From named to unnamed

To translate from named schemas/tuples/relations to unnamed ones, we need to *fix an order* of the attributes.

So for the entire lecture we fix a global *linear order* $\lessdot$ *on Att*.

For example, if Code $\lessdot$ Symb $\lessdot$ Name then

Currencies

Name	Code	Symb
Euro	EUR	€
Canadian dollar	CAD	$
Cuban peso	CUP	$
Indian rupee	INR	₹
Sterling	GBP	£

=

Currencies

Code	Symb	Name
EUR	€	Euro
CAD	$	Canadian dollar
CUP	$	Cuban peso
INR	₹	Indian rupee
GBP	£	Sterling

columns reordered using $\lessdot$

⇝

Currencies

EUR	€	Euro
CAD	$	Canadian dollar
CUP	$	Cuban peso
INR	₹	Indian rupee
GBP	£	Sterling

unnamed relation

From unnamed to named

slide 3.6

To translate from unnamed to named, we *assume* $1, 2, 3, \ldots \in Att$ and introduce attribute names according to the column order:

Currencies

EUR	€	Euro
CAD	$	Canadian dollar
CUP	$	Cuban peso
INR	₹	Indian rupee
GBP	£	Sterling

unnamed relation

⇝

Currencies

1	2	3
EUR	€	Euro
CAD	$	Canadian dollar
CUP	$	Cuban peso
INR	₹	Indian rupee
GBP	£	Sterling

named relation

3.2 Unnamed RA

Unnamed Relational Algebra: k-conditions (syntax)

slide 3.7

A *k-condition* is a Boolean combination of statements of the form $i \doteq j$ or $i \doteq a$ for $i, j \in [k]$ and $a \in Const$.

similar to "condition over U" (see named RA) with numbers instead of attributes

Recursive definition:

Base cases

- $i \doteq j$ is a k-condition for all $i, j \in [k]$.
- $i \doteq a$ is a k-condition for all $i \in [k]$ and $a \in Const$.

Recursive cases

- If θ is a k-condition, then $\neg\theta$ is a k-condition.
- If θ_1 and θ_2 are k-conditions, then $(\theta_1 \vee \theta_2)$ is a k-condition.
- If θ_1 and θ_2 are k-conditions, then $(\theta_1 \wedge \theta_2)$ is a k-condition.

Unnamed Relational Algebra: k-conditions (semantics)

slide 3.8

For every k-condition θ and k-tuple $\overline{a} = (a_1, \ldots, a_k)$ we define the relation *"$\overline{a}$ satisfies θ ($\overline{a} \models \theta$)"*:

Base cases

- $\overline{a} \models i \doteq j$ if $a_i = a_j$.
- $\overline{a} \models i \doteq a$ if $a_i = a$.

Recursive cases

- $\overline{a} \models \neg\theta$ if it is not the case that $\overline{a} \models \theta$
- $\overline{a} \models (\theta_1 \vee \theta_2)$ if $\overline{a} \models \theta_1$ or $\overline{a} \models \theta_2$ (or both)
- $\overline{a} \models (\theta_1 \wedge \theta_2)$ if $\overline{a} \models \theta_1$ and $\overline{a} \models \theta_2$

Unnamed Relational Algebra: Syntax

slide 3.9

Let $\boldsymbol{S}$ be a unnamed schema. We recursively define *uRA expressions of arity k (over $\boldsymbol{S}$)*:

Base cases

- R is an uRA-expression of arity $\boldsymbol{S}(R)$ for all $R \in dom(\boldsymbol{S})$.
- $\{a\}$ is an uRA-expression of arity 1 for all $a \in Const$.

Recursive cases

- **(selection)** If e is an uRA-expression of arity k and θ is a k-condition, then $\sigma_\theta(e)$ is an uRA-expression of arity k.
- **(projection)** If e is an uRA-expression of arity k and $\alpha = (i_1, \ldots, i_m)$ is a list of numbers $1 \leqslant i_\ell \leqslant k$, then $\pi_\alpha(e)$ is an uRA-expression of arity m.

- **(Cartesian product)** If e_1 and e_2 are uRA-expressions of sort k_1 and k_2 (resp.), then $(e_1 \times e_2)$ is an uRA-expression of arity $k_1 + k_2$.
- **(union / difference)** If e_1 and e_2 are uRA-expressions of arity k, then $(e_1 \cup e_2)$ and $(e_1 - e_2)$ are uRA-expressions of arity k.

slide 3.10

Unnamed Relational Algebra: Semantics

Let $\boldsymbol{S}$ be an unnamed schema. For every k-ary RA expression e we recursively define the k-ary query function $q_e(D)$ applied to database D over $\boldsymbol{S}$:

Base cases

- If $e = R$, then $q_e(D) := D(R)$.
- If $e = \{a\}$, then $q_e(D) := \{a\}$.

Recursive cases

- If $e = \sigma_\theta(e')$, then $q_e(D) := \{\overline{a} \in q_{e'}(D) \mid \overline{a} \vDash \theta\}$.
- If $e = \pi_\alpha(e')$ and $\alpha = (i_1, \dots, i_m)$, then $q_e(D) := \{(a_{i_1}, \dots, a_{i_m}) \mid (a_1, \dots, a_k) \in q_{e'}(D)\}$.
- If $e = (e_1 \times e_2)$, then
  $q_e(D) := \{(a_1, \dots, a_{k_1}, b_1, \dots, b_{k_2}) \mid (a_1, \dots, a_{k_1}) \in q_{e_2}(D) \text{ and } (b_1, \dots, b_{k_2}) \in q_{e_2}(D)\}$.
- If $e = (e_1 \cup e_2)$, then $q_e(D) := q_{e_1}(D) \cup q_{e_2}(D)$.
- If $e = (e_1 - e_2)$, then $q_e(D) := q_{e_1}(D) \setminus q_{e_2}(D)$.

slide 3.11

Equivalence between named and unnamed RA

For *named* schemas $\boldsymbol{S}$, database relations R, and databases D, let $\widetilde{\boldsymbol{S}}$, $\widetilde{R}$, and $\widetilde{D}$ be the corresponding *unnamed* version (as discussed before).

Lemma 3.1. Let $\boldsymbol{S}$ be a named schema. For every RA-expression e over $\boldsymbol{S}$ there is an uRA-expression $\tilde{e}$ over $\widetilde{\boldsymbol{S}}$ such that for every database D over $\boldsymbol{S}$: $q_{\tilde{e}}(\widetilde{D}) = \widetilde{q_e(D)}$.

Similarly, for *unnamed* schemas $\boldsymbol{S}$, database relations R, and databases D, let $\widehat{\boldsymbol{S}}$, $\widehat{R}$, and $\widehat{D}$ be the corresponding *named* version.

Lemma 3.2. Let $\boldsymbol{S}$ be an unnamed schema. For every uRA-expression e over $\boldsymbol{S}$ there is a RA-expression $\hat{e}$ over $\widehat{\boldsymbol{S}}$ such that for every database D over $\boldsymbol{S}$: $q_{\hat{e}}(\widehat{D}) = \widehat{q_e(D)}$.

Both lemmas imply that RA and uRA are equally expressive:

Theorem 3.3. RA $\equiv$ uRA.

slide 4.1

4 First-order logic

slide 4.2

Variables and atoms

Recall, that we have global sets of relation names *Rel* and values/"constants" *Const*.

In first-order logic, we additionally need *variables*:

- Fix an infinite set *Var* of variables disjoint from *Rel*, *Const*, *Att*.
- Usually denoted by x, y, z and variants.
- These variables will later take values from *Const*.

slide 4.3

First-order logic: syntax

Let $\boldsymbol{S}$ be an unnamed schema. We recursively define *first-order formulas over schema* $\boldsymbol{S}$ (just "formula" in the following):

Base cases (also called atomic formulas)

- $u_1 = u_2$ is a formula for all $u_1, u_2 \in \mathit{Var} \cup \mathit{Const}$.[2]
- $R(u_1, \dots, u_k)$ is a formula for all $R \in \mathit{dom}(\boldsymbol{S})$ with $\boldsymbol{S}(R) = k$ and all $u_i \in \mathit{Var} \cup \mathit{Const}$.

Recursive cases

- If φ and ψ are formulas, then $(\varphi \wedge \psi)$ is a formula.
- If φ and ψ are formulas, then $(\varphi \vee \psi)$ is a formula.
- If φ is a formula, then $\neg\varphi$ is a formula.
- If φ is a formula and $x \in \mathit{Var}$, then $\exists x\, \varphi$ is a formula.
- If φ is a formula and $x \in \mathit{Var}$, then $\forall x\, \varphi$ is a formula.

We use $\varphi \to \psi$ as abbreviation for $\neg\varphi \vee \psi$
and $\varphi \leftrightarrow \psi$ as abbreviation for $(\varphi \to \psi) \wedge (\psi \to \varphi)$.

Free variables and assignments

slide 4.4

In FO one can not only express *sentences* (which are either true or false on a database) but also *define relations* using free variables.

We recursively define the set $\mathit{free}(\varphi)$ of free variables in φ.

Base cases

- $\mathit{free}(u_1 = u_2) := \{u_1, u_2\} \cap \mathit{Var}$
- $\mathit{free}\big(R(u_1, \dots, u_k)\big) := \{u_1, \dots, u_k\} \cap \mathit{Var}$

Recursive cases

- $\mathit{free}(\varphi \wedge \psi) := \mathit{free}(\varphi) \cup \mathit{free}(\psi)$ and $\mathit{free}(\varphi \vee \psi) := \mathit{free}(\varphi) \cup \mathit{free}(\psi)$
- $\mathit{free}(\neg\varphi) := \mathit{free}(\varphi)$
- $\mathit{free}(\exists x\, \varphi) := \mathit{free}(\varphi) \setminus \{x\}$ and $\mathit{free}(\forall x\, \varphi) := \mathit{free}(\varphi) \setminus \{x\}$

An *assignment for* φ is a mapping $\beta : \mathit{free}(\varphi) \to \mathit{Const}$.

Whenever necessary we silently extend β to an identity on Const, that is, $\beta : \mathit{free}(\varphi) \cup \mathit{Const} \to \mathit{Const}$ with $\beta(a) := a$ for all $a \in \mathit{Const}$.

Active domains

slide 4.5

Remark 4.1. There are several slightly different variants of defining semantics of first-order logic as a database query language. (See also Section 5.3 and 5.4 in [AHV95].) Here we just introduce one of them: *active domain semantics*.

Definition 4.2.

1. The active domain $\mathit{adom}(D) \subseteq \mathit{Const}$ of a database D is the finite set of constants that appear in some relation of D.
2. The active domain $\mathit{adom}(\varphi) \subseteq \mathit{Const}$ of a first-order query φ is the finite set of constants that appear in φ.

Relativized semantics of first-order formulas

slide 4.6

Fix an unnamed relational schema $\boldsymbol{S}$ and a set of constants $A \subseteq \mathit{Const}$.

For all

- databases D over schema $\boldsymbol{S}$ with $\mathit{adom}(D) \subseteq A$,
- first-order formulas φ over schema $\boldsymbol{S}$ with $\mathit{adom}(\varphi) \subseteq A$, and
- assignments $\beta : \mathit{free}(\varphi) \to A$ to the free variables of φ

we define the relativized semantics

[2] Note that this in particular allows formulas of the form $x = y$ for $x, y \in \mathit{Var}$ articulating that both variables shall have the same value as well as $x = a$ for $x \in \mathit{Var}$ and $a \in \mathit{Const}$ articulating that x shall have the value a. Moreover, we also allow "$a = a$" and "$a = b$" for distinct constants $a, b \in \mathit{Const}$, but we only include this for technical reasons here; "$a = a$" will always be true and "$a = b$" will always be false (for distinct a, b).

$$(D, \beta) \vDash_A \varphi$$

"φ is true in D under β relative to A"

recursively over the definition of φ.

slide 4.7

Base cases

- $(D, \beta) \vDash_A u_1 = u_2$ if $\beta(u_1) = \beta(u_2)$
- $(D, \beta) \vDash_A R(u_1, \dots, u_k)$ if $\big(\beta(u_1), \dots, \beta(u_k)\big) \in D(R)$

Recursive cases

- $(D, \beta) \vDash_A (\varphi \wedge \psi)$ if $(D, \beta) \vDash_A \varphi$ and $(D, \beta) \vDash_A \psi$
- $(D, \beta) \vDash_A (\varphi \vee \psi)$ if $(D, \beta) \vDash_A \varphi$ or $(D, \beta) \vDash_A \psi$ (or both)
- $(D, \beta) \vDash_A \neg\varphi$ if it is not the case that $(D, \beta) \vDash_A \varphi$
- $(D, \beta) \vDash_A \exists x\, \varphi$ if *there is an* $a \in A$ such that $(D, \beta \cup \{x \mapsto a\}) \vDash_A \varphi$
- $(D, \beta) \vDash_A \forall x\, \varphi$ if *for all* $a \in A$ it holds that $(D, \beta \cup \{x \mapsto a\}) \vDash_A \varphi$

We write $(D, \beta) \vDash \varphi$ as an abbreviation for $(D, \beta) \vDash_A \varphi$ with $A = adom(D) \cup adom(\varphi)$.

slide 4.8

First-order queries

A k-ary ***first-order query*** $\varphi(x_1, \dots, x_k)$ consists of

- a first-order formula φ and
- a k-tuple $(x_1, \dots, x_k)$ containing all free variables from φ, i. e., $\{x_1, \dots, x_k\} = free(\varphi)$.

A first-order query $\varphi(x_1, \dots, x_k)$ defines the query function

(1) $$q_{\varphi(x_1,\dots,x_k)}(D) := \big\{ \big(\beta(x_1), \dots, \beta(x_k)\big) \mid \beta : free(\varphi) \to adom(D) \cup adom(\varphi) \text{ such that } (D, \beta) \vDash \varphi \big\}$$

4.1 First-order logic and relational algebra

slide 4.9

Some notation: substitution

Let φ be a first-order formula, $x \in Var$, and $u \in Var \cup Const$.
By $\varphi[x \rightsquigarrow u]$ we denote the formula where every *free* occurrence of x is replaced by u:

Base cases

- $u_1 = u_2[x \rightsquigarrow u] := u_1' = u_2'$ and $R(u_1, \dots, u_k)[x \rightsquigarrow u] := R(u_1', \dots, u_k')$ where

$$u_i' := \begin{cases} u, & \text{if } u_i = x \\ u_i, & \text{if } u_i \neq x \end{cases}$$

Recursive cases

- $(\varphi \wedge \psi)[x \rightsquigarrow u] := (\varphi[x \rightsquigarrow u] \wedge \psi[x \rightsquigarrow u])$ and $(\varphi \vee \psi)[x \rightsquigarrow u] := (\varphi[x \rightsquigarrow u] \vee \psi[x \rightsquigarrow u])$
- $(\neg\varphi)[x \rightsquigarrow u] := \neg(\varphi[x \rightsquigarrow u])$
- $(\exists y\, \varphi)[x \rightsquigarrow u] := \begin{cases} \exists y\, \varphi, & \text{if } y = x \\ \exists y\, (\varphi[x \rightsquigarrow u]), & \text{if } y \neq x \end{cases}$
- $(\forall y\, \varphi)[x \rightsquigarrow u] := \begin{cases} \forall y\, \varphi, & \text{if } y = x \\ \forall y\, (\varphi[x \rightsquigarrow u]), & \text{if } y \neq x \end{cases}$

slide 4.10

From uRA to FO

slide 4.11

From FO to uRA

References

[AHV95] S. Abiteboul, R. Hull, and V. Vianu. *Foundations of Databases*. Addison-Wesley, 1995. URL: http://webdam.inria.fr/Alice/ (cit. on pp. 3, 15).

[ABLMP22] M. Arenas, P. Barceló, L. Libkin, W. Martens, and A. Pieris. *Database Theory*. Open access at https://github.com/pdm-book/community, 2022 (cit. on p. 3).